zkDietFormer

Anonymous Author(s)

Abstract

Keywords

Zero-Knowledge Proofs

1 Introduction

Recent breakthroughs in Transformer-based models have revolutionized artificial intelligence, facilitating the widespread deployment of high-performance Large Language Models (LLMs) via Model-as-a-Service (MaaS) platforms. In this setting, model providers typically large models, while users receive outputs through APIs or remote inference interfaces. However, this architecture inherently obscures the generation process: users cannot independently verify whether the received output truly produced from the specific model architecture and weight parameters intended. Users are therefore forced to rely on trust regarding whether the provider substituted a cheaper model for the same input, post-processed or modified outputs due to internal policies, or unintentionally changed the model during updates or deployment. This problem of model execution integrity is particularly critical in high-stakes domains such as finance, healthcare, and security, and becomes increasingly central as demands for transparency and accountability in inference continue to grow.

One may attempt to address this trust issue by requiring the model provider to disclose model parameters for verification. In practice, however, this is practically infeasible. Model weights are the result of substantial investments in data, compute, and engineering, and are protected as proprietary intellectual property (IP). Consequently, a fundamental dilemma arises: users demand integrity guarantees for the model's outputs, while providers must preserve the privacy of their model parameters to protect their assets.

Zero-knowledge proofs (ZKPs) provide a principled way to reconcile these conflicting requirements. By leveraging ZKPs, a prover (the model provider) can cryptographically prove that an output y is the result of evaluating the model on the user's input x with a secret weight W, i.e., $f(x, W) = y$, without revealing any information about W itself. This enables verifiable inference while maintaining the confidentiality of model parameters.

However, making ZKP-based verifiable AI practically deployable requires the efficient proving of computationally intensive deep learning operations. In modern Transformer architectures, the primary bottleneck is the self-attention mechanism, the computational core of the model. For an input sequence of length n and hidden dimension d, the input $X \in \mathbb{R}^{n\times d}$ is projected into queries, keys, and values $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{n\times d}$ (for a single attention head). Standard self-attention forms an attention matrix $\mathbf{QK}^\top \in \mathbb{R}^{n\times n}$, applies a row-wise softmax over the entire matrix, and then multiplies by $\mathbf{V}$ to obtain the attention output.

This computation requires $O(n^2 d)$ linear operations for matrix multiplications, as well as an additional $O(n^2)$ nonlinear cost induced by the softmax. The resulting computational overhead is substantial, posing the single largest obstacle to proving large-scale Transformer models.

The most naïve approach to proving self-attention involves arithmetizing the full $\mathbf{QK}^\top$ computation into a circuit and enforcing the resulting $O(n^2 d)$ operations within a SNARK. However, this cost far exceeds practical limits. To mitigate this, many systems have adopted probabilistic verification techniques for large matrix products, such as Freivalds' algorithm [11] or Thaler's randomized checks [23]. These methods verify the $n \times n$ multiplication relation without full recomputation by sampling random vectors, compressing verification to a small number of checks and reducing the cost to $O(n^2)$. Nevertheless, the complexity remains dominated by the quadratic term, which becomes increasingly prohibitive as models scale. For instance, while BERT [9] uses 512 tokens and GPT-3 [2] uses a 2,048-token context window, frontier models now support contexts in the hundreds of thousands or even millions (e.g., 400K for GPT-5 [19] and 10M for Llama 4 Scout [18]). As n grows, this quadratic verification cost renders proof generation for realistic sequence lengths impractical.

Meanwhile, deep learning literature has repeatedly demonstrated that attention matrices often exhibit an approximate low-rank structure. Concretely, when the number of landmarks $m \ll n$, the attention matrix can be effectively approximated by a rank-m representation, enabling training and inference costs to approach linearity with respect to the sequence length [25, 29]. Despite these advances, the field of Verifiable AI has yet to exploit low-rank structures as a primary mechanism for reducing proof generation costs. Consequently, a gap persists: while low-rank approximations improve computational efficiency in practice, the lack of proof systems tailored to these structures keeps proof complexity anchored at $O(n^2)$. This work aims to bridge this gap. Our central idea is to combine low-rank structures with probabilistic verification to eliminate the quadratic complexity in verifying self-attention matrices.

1.1 Our Contribution

In this work, we propose **zkDietFormer**, a verification protocol that eliminates the dominant $O(n^2)$ bottleneck in proving self-attention by combining a low-rank representation of the attention matrix, Freivalds' randomized checks, and a commit-carrying SNARK. Our main contributions are summarized as follows:

- **The first verification framework tailored to low-rank self-attention.** To the best of our knowledge, we present the first verification framework specifically designed for

This work is licensed under the Creative Commons Attribution 4.0 International License. To view a copy of this license visit https://creativecommons.org/licenses/by/4.0/ or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

Proceedings on Privacy Enhancing Technologies YYYY(X), 1–15

© YYYY Copyright held by the owner/author(s).

https://doi.org/XXXXXXX.XXXXXXX

low-rank self-attention. We instantiate this framework using the Nyström method, reducing the overall circuit complexity—including both the core matrix operations of self-attention and the nonlinear operations induced by row-wise softmax—to $O(n)$.

- **Constraint-efficient verification via Freivalds' checks and CC-SNARKs.** We employ Freivalds' algorithm to efficiently verify large matrix computations inside the circuit, while leveraging commit-carrying SNARKs to offload computationally expensive operations outside the circuit, significantly reducing the total number of constraints.
- **Practical implementation and evaluation.** We implemented a prototype of the proposed framework and evaluated its performance through benchmarks. The results demonstrate a significant reduction in prover costs for practical sequence lengths, showing that **zkDietFormer**is not only theoretically efficient but also practically deployable.

In summary, the prover's constraint cost in **zkDietFormer**scales linearly with the input sequence length n. Consequently, to the best of our knowledge, this work provides the first verifiable mechanism for low-rank self-attention that enables proof generation with cost linear to the sequence length.

2 Related Works

In this section, we provide a brief overview of the core operations of Transformer models and review prior work aimed at improving the efficiency of matrix multiplication, which is widely recognized as a major computational bottleneck. We also examine existing approaches for constructing zero-knowledge proofs for matrix multiplication.

Low-Rank Transformer The Transformer is a deep learning architecture proposed by Google in 2017 [24], which leverages the attention mechanism as its core component to effectively model relationships within sequential data, such as words in a sentence. The core computation of this model consists of matrix multiplications among the query, key, and value matrices. In standard Transformers, these operations incur quadratic computational complexity with respect to the input sequence length n, causing the computational cost to grow rapidly as n increases. To mitigate this bottleneck, Child et al. [6] proposed the Sparse Transformer, which restricts attention to predefined sparse patterns, using strided attention for data with periodic structures such as images and audio, and fixed attention for text without structure. However, this approach relies on implementations tied to specific TensorFlow versions. To address this limitation, Longformer [1] introduced sliding-window attention, where each token attends only to nearby tokens, providing a more general and maintainable solution. Rae et al. [20] proposed the Compressive Transformer, which compresses past hidden activations(memories) using pooling operations and jointly attends to both recent memories and compressed long-term memories to efficiently model long-range context. Nevertheless, these approaches do not preserve full attention, leading to limitations such as insufficient modeling of long-range dependencies [1, 6], or performance degradation in exchange for computational efficiency [1]. Accordingly, Linformer [25] shows that the attention matrix induced by self-attention exhibits a low-rank structure both theoretically and empirically, and approximates scaled dot-product attention by projecting the key and value matrices into a lower-dimensional space via linear projections. As a result, the time and space complexity of self-attention are reduced to linear in the sequence length n. However, Linformer has limitations in causal modeling, where future tokens must be masked, and can incur large mean squared errors compared to exact attention. To overcome these issues, Performer [7] introduces a linear-complexity attention mechanism that avoids explicit projection matrices and low-rank assumptions, and instead approximates full softmax attention via FAVOR+, which is based on positive orthogonal random features for softmax kernel estimation. Nyströmformer [29] introduces a Nyström low-rank approximation that reconstructs the full attention map using a subset of landmark tokens, achieving higher accuracy than Performer while maintaining linear complexity with respect to the sequence length.

zk-SNARKs for Matrix Multiplication There has been ongoing research on verifying the integrity of matrix multiplication using zero-knowledge proofs. Since convolution operations can be reduced to structured matrix multiplication, such verification techniques have been extensively studied as a core technique for proving the correctness and integrity of AI model execution.

Thaler et al. [23] introduced the reuse-of-work technique into the Sumcheck protocol for matrix multiplication verification, reducing the prover's computational overhead from $O(n^3)$ to $O(n^2)$ for $n \times n$ matrices. zkCNN [17] adopts this optimized Sumcheck protocol to efficiently verify matrix multiplication in fully connected layers. pvCNN [26] transforms convolution operations, which inherently require a large number of multiplication gates, into a single matrix multiplication by reshaping the filters and inputs into rectangular matrices. Inspired by the QPP (Quadratic Polynomial Programs) approach of vCNN [16], which applies polynomial representations within the QAP framework, pvCNN further extends this idea by introducing the QMP (Quadratic Matrix Program). As a result, the convolution can be expressed and verified using only a single multiplication gate. zkML [5] applies Freivalds' algorithm [11] to verify matrix multiplication operations in linear layers, such as convolutions and batched matrix multiplications, achieving verification complexity of $O(n^2)$. zkLLM [22] employs specialized sumcheck protocols for tensor operations, achieving prover complexity linear in the total tensor size $O(D)$, together with tlookup and zkAttn for non-arithmetic operations such as softmax. This line of work builds on the interactive-proof approach to neural network verification introduced by SafetyNets [12], which applies GKR-based protocols to matrix multiplication in arithmetic-circuit-representable networks. zkMatrix [8] applies Bulletproofs-based acceleration to committed matrix multiplication, achieving $O(n^2)$ prover time with $O(\log n)$ verifier time and transcript size for $n \times n$ matrices, and reducing the marginal cost of each additional multiplication to $O(n)$ group operations under batching.

However, these approaches still require a prover computation complexity of $O(n^2)$ with respect to the matrix dimension (or sequence length) [5, 8, 16, 17, 23], or, even when achieving linear-time complexity in theory, suffer from the limitation that the other matrix dimension m remains prohibitively large in practice [22]. Moreover, nonlinear functions such as softmax cannot be directly handled within ZKP circuits and thus must be approximated, which leads to

accuracy degradation and necessitates the design of model architectures specifically tailored for zero-knowledge proof systems [30].

3 Preliminaries

Notation. We denote the sequence length by n, the hidden dimension by d, and the per-head query/key and value dimensions by d_k and d_v, respectively. The number of attention heads is h, the number of Transformer layers is L, and the number of landmarks is m, giving a segment length $\ell = n/m$. The circuit precision parameter k_{bits} controls the bit-width of quantized fixed-point arithmetic.

3.1 Scaled Dot-Product Attention

The model represents interactions between tokens using queries and keys of dimension d_k, and values of dimension d_v. For each token, its associated query computes its similarity scores with the keys of all other tokens in the sequence, thereby determining which parts of the input the model should attend to more strongly. The contextual information is then aggregated by computing a weighted sum of the corresponding value vectors based on these similarities.

The attention mechanism is defined as a function that takes a single query together with a set of key–value pairs and produces an output vector, where the query, keys, values, and outputs all lie in vector spaces. In practice, multiple queries are processed simultaneously by stacking them into matrices $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$. These matrices are obtained by projecting the input matrix $\mathbf{X} \in \mathbb{R}^{n\times d}$ using learnable parameter matrices $\mathbf{W}^Q \in \mathbb{R}^{d\times d_k}$, $\mathbf{W}^K \in \mathbb{R}^{d\times d_k}$, and $\mathbf{W}^V \in \mathbb{R}^{d\times d_v}$, yielding $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q \in \mathbb{R}^{n\times d_k}$, $\mathbf{K} = \mathbf{X}\mathbf{W}^K \in \mathbb{R}^{n\times d_k}$, and $\mathbf{V} = \mathbf{X}\mathbf{W}^V \in \mathbb{R}^{n\times d_v}$. Concretely, the attention weights are obtained by scaling the dot product between queries and keys by $1/\sqrt{d_k}$, followed by the application of the softmax function. The final output is computed as the weighted sum of the value vectors using these attention weights.

The output matrix is computed as follows:

$$\text{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

3.2 Multihead Attention

Multi-head attention replaces a single attention function over d-dimensional queries, keys, and values with h parallel attention heads. The queries, keys, and values are first linearly projected h times into lower-dimensional subspaces of sizes d_k and d_v, respectively. For each head i,

$$\text{head}_i = \text{Attention}(\mathbf{Q}_i,\mathbf{K}_i,\mathbf{V}_i)$$

where $\mathbf{Q}_i = \mathbf{X}\mathbf{W}_i^Q$, $\mathbf{K}_i = \mathbf{X}\mathbf{W}_i^K$, $\mathbf{V}_i = \mathbf{X}\mathbf{W}_i^V$, with learnable projection matrices $\mathbf{W}_i^Q \in \mathbb{R}^{d\times d_k}$, $\mathbf{W}_i^K \in \mathbb{R}^{d\times d_k}$, and $\mathbf{W}_i^V \in \mathbb{R}^{d\times d_v}$. The resulting head outputs are concatenated and linearly transformed to form the final representation:

$$\text{MultiHead}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{Concat}(\text{head}_1,\ldots,\text{head}_h)\mathbf{W}^O$$

with $\mathbf{W}^O \in \mathbb{R}^{hd_v\times d}$. In the standard setting, $d_k = d_v = d/h$, so that the total cost across heads matches that of single-head attention over d dimensions. This design enables the model to attend to diverse representation subspaces while maintaining computational cost comparable to single-head attention. For clarity, we describe the protocol for a single attention head with $d_k = d_v = d$; the extension to h heads follows by applying the same construction per head.

3.3 Nystrom-Based Linear Attention

Nyströmformer applies the Nyström method [29]—a technique widely used in matrix approximation—to reduce the computational complexity of the standard Transformer from $O(n^2)$ to $O(n)$. The key idea of the Nyström method is to reconstruct the full matrix approximately by using only a subset of its rows and columns, instead of computing the entire matrix explicitly.

When the softmax matrix S is expressed as a block matrix as shown below,

$$S = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) = \begin{bmatrix} A_S & B_S \\ F_S & C_S \end{bmatrix}$$

the Nystrom method approximates the expensive block $C_S \in \mathbb{R}^{(n-m)\times(n-m)}$ by using the smaller blocks $A_S \in \mathbb{R}^{m\times m}, B_S \in \mathbb{R}^{m\times(n-m)}, F_S \in \mathbb{R}^{(n-m)\times m}$ as illustrated below:

$$\hat{S} = \begin{bmatrix} A_S & B_S \\ F_S & F_S A_S^+ B_S \end{bmatrix} = \begin{bmatrix} A_S \\ F_S \end{bmatrix} A_S^+ \begin{bmatrix} A_S & B_S \end{bmatrix}$$

Here, A_S^+ denotes the Moore-Penrose pseudoinverse of A_S.

However, when this approach is applied directly to self-attention, the row-wise normalization property of the softmax function still requires accessing all entries of the $n\times n$ matrix in order to compute the denominators. To address this issue, Nyströmformer partitions the entire sequence into m segments of length $\ell = n/m$ before applying softmax, and computes the mean within each segment to obtain m landmark points for the queries and keys, respectively. These landmarks yield $\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}} \in \mathbb{R}^{m\times d}$, from which three low-rank factors $\mathbf{Q}\widetilde{\mathbf{K}}^\top \in \mathbb{R}^{n\times m}$, $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top \in \mathbb{R}^{m\times m}$, and $\widetilde{\mathbf{Q}}\mathbf{K}^\top \in \mathbb{R}^{m\times n}$ are formed. The resulting factors play the same structural role as $\begin{pmatrix} A_S \\ F_S \end{pmatrix}$, A_S^+, and $\begin{pmatrix} A_S & B_S \end{pmatrix}$ in the block decomposition above, but are computed from landmarks rather than extracted from S. Using these matrices, Nyströmformer derives a final approximation that achieves linear computational complexity:

$$\hat{S}V = \text{softmax}\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)\text{softmax}\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)^+\text{softmax}\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\right)V$$

The exact computation of the pseudoinverse A_S^+ (e.g., via SVD) is computationally expensive and poorly suited for GPU execution, becoming a practical bottleneck in implementation. To mitigate this issue, Nyströmformer introduces an iterative algorithm that approximates the pseudoinverse using repeated matrix multiplications. The resulting approximation Z^* is then substituted into the final formulation, significantly improving computational efficiency.

$$\hat{S}V = \text{softmax}\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)Z^*\,\text{softmax}\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\right)V$$

3.4 zk-SNARKs

A *zero-knowledge succinct non-interactive argument of knowledge*(zk-SNARKs) for an NP relation $\mathcal{R}$ consists of three PPT algorithms Π = (Setup, Prove, Verify):

- $(\mathsf{pk}, \mathsf{vk}) \leftarrow \Pi.\mathsf{Setup}(1^\lambda, \mathcal{R})$: On input the security parameter λ and a relation $\mathcal{R}$, the setup algorithm outputs a proving key pk and a verification key vk.
- $\pi \leftarrow \Pi.\mathsf{Prove}(\mathsf{pk}, \mathbf{x}; \mathbf{w})$: On input the proving key pk, a statement $\mathbf{x}$, and a witness $\mathbf{w}$ such that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, the prover outputs a proof π.
- $0/1 \leftarrow \Pi.\mathsf{Verify}(\mathsf{vk}, \mathbf{x}, \pi)$: On input the verification key vk, a statement $\mathbf{x}$, and a proof π, the verifier outputs 0(reject) or 1(accept)

A zk-SNARK system Π satisfies the following properties:
Completeness. For every true statement for the relation $\mathcal{R}$, an honest prover holding a valid witness always convinces the verifier, i.e., $\forall(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$:

$$\Pr\left[\begin{array}{l}(\mathsf{pk}, \mathsf{vk}) \leftarrow \Pi.\mathsf{Setup}(1^\lambda, \mathcal{R}) \\ \pi \leftarrow \Pi.\mathsf{Prove}(\mathsf{pk}, \mathbf{x}; \mathbf{w})\end{array} : \quad \Pi.\mathsf{Verify}(\mathsf{vk}, \mathbf{x}, \pi) = 1\right] = 1$$

Knowledge Soundness. Knowledge soundness requires that any prover producing a proof π that makes the verifier accept must possess a corresponding witness, and that a knowledge extractor $\mathcal{E}$ can efficiently extract such a witness from π. Formally, for any PPT adversary $\mathcal{A}$, the following is *negligible*.

$$\Pr\left[\begin{array}{ll}(\mathsf{pk}, \mathsf{vk}) \leftarrow \Pi.\mathsf{Setup}(\mathcal{R}) & \Pi.\mathsf{Verify}(\mathsf{vk}, \mathbf{x}^*, \pi^*) = 1 \\ (\mathbf{x}^*, \pi^*) \leftarrow \mathcal{A}(\mathsf{pk}, \mathsf{vk}) & : \ \wedge \\ \mathbf{w} \leftarrow \mathcal{E}_{\mathcal{A}}(trans_{\mathcal{A}}) & (\mathbf{x}^*; \mathbf{w}) \notin \mathcal{R}\end{array}\right]$$

Zero-Knowledge. There exists a PPT simulator $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ such that $\mathcal{S}_1$ outputs simulated setup parameters (pk, vk) together with a trapdoor td. On input $(\mathsf{pk}, \mathsf{vk}, td)$, $\mathcal{S}_2$ outputs a simulated proof π. Moreover, for all PPT adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, we have that

$$\left|\Pr\left[\mathcal{A}_2(\mathsf{vk}, \mathbf{x}, \pi) = 1 : \begin{array}{l}(\mathsf{pk}, \mathsf{vk}) \leftarrow \Pi.\mathsf{Setup}(1^\lambda, \mathcal{R}) \\ (\mathbf{x}, \mathbf{w}) \leftarrow \mathcal{A}_1(1^\lambda, \mathsf{pk}, \mathsf{vk}) \\ \pi \leftarrow \Pi.\mathsf{Prove}(\mathsf{pk}, \mathbf{x}; \mathbf{w})\end{array}\right]\right.$$
$$\left. - \Pr\left[\mathcal{A}_2(\mathsf{vk}, \mathbf{x}, \pi) = 1 : \begin{array}{l}(\mathsf{pk}, \mathsf{vk}, td) \leftarrow \mathcal{S}_1(1^\lambda, \mathcal{R}) \\ (\mathbf{x}, \mathbf{w}) \leftarrow \mathcal{A}_1(1^\lambda, \mathsf{pk}, \mathsf{vk}) \\ \pi \leftarrow \mathcal{S}_2(\mathsf{pk}, \mathbf{x}, td)\end{array}\right]\right| \leq \mathsf{negl}(\lambda).$$

Succinctness. For any statement $\mathbf{x}$ and witness $\mathbf{w}$, the size of proof π satisfies $|\pi| = poly(\lambda) \cdot polylog(|\mathbf{x}| + |\mathbf{w}|)$

Commit-and-Prove SNARKs (CP-SNARKs). A *Commit-and-Prove SNARKs* (CP-SNARKs) [3, 4] allow a prover to prove statements about values hidden in a commitment. Let Com be a commitment scheme. Given a commitment cm to $\mathbf{u}$, a CP-SNARK proves knowledge of a witness $\mathbf{w}$ consistent with cm and satisfying a relation $\mathcal{R}$. Formally, for $\mathcal{R}(\mathbf{x}; \mathbf{w})$, we write $\mathbf{w} := (\omega, \mathbf{u}, o)$, where $\mathbf{u}$ is committed, ω is uncommitted, and o is the opening randomness, such that $\mathsf{cm} = \mathsf{Com}(\mathbf{u}; o)$ and $\mathcal{R}(\mathbf{x}; \mathbf{w}) = 1$. A CP-SNARK is a tuple of algorithms $\Pi_{\mathsf{cp}} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$.

- $(\mathsf{pk}, \mathsf{vk}) \leftarrow \Pi_{\mathsf{cp}}.\mathsf{Setup}(1^\lambda, \mathcal{R}, \mathsf{ck})$: Same as $\Pi.\mathsf{Setup}$, but additionally takes the commitment key ck as input.
- $\pi_{\mathsf{cp}} \leftarrow \Pi_{\mathsf{cp}}.\mathsf{Prove}(\mathsf{pk}, \mathbf{x}; \mathbf{w})$: Equal to $\Pi.\mathsf{Prove}(\mathsf{pk}, \{\mathbf{x}, \mathsf{cm}\}; \{\mathbf{u}, o\}, \omega)$. It is based on $\Pi.\mathsf{Prove}$, but additionally takes the commitment cm as input, and the witness includes the committed value $\mathbf{u}$ and its opening o, together with the uncommitted part ω.
- $0/1 \leftarrow \Pi_{\mathsf{cp}}.\mathsf{Verify}(\mathsf{vk}, \mathbf{x}, \mathsf{cm}, \pi_{\mathsf{cp}})$ Same as $\Pi.\mathsf{Verify}$, except that it additionally takes cm as input.

Commit-Carrying SNARKs (CC-SNARKs). A *Commit-Carrying SNARKs* (CC-SNARKs) [4, 14] extend the commit-and-prove approach by generating the witness commitment during proving. This helps scale to large circuits by moving expensive computations (e.g., group exponentiation) outside the circuit: the prover commits using a circuit-dependent key ck, and the resulting commitment cm is carried into the verification equation. A cc-SNARK consists of three PPT algorithms $\Pi_{\mathsf{cc}} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$.

- $(\mathsf{ck}, \mathsf{pk}, \mathsf{vk}) \leftarrow \Pi_{\mathsf{cc}}.\mathsf{Setup}(1^\lambda, \mathcal{R})$: Similar to $\Pi_{\mathsf{cp}}.\mathsf{Setup}$, but additionally outputs the circuit-dependent commitment key ck together with (pk, vk).
- $(\pi_{\mathsf{cc}}, \tilde{\mathsf{cm}}) \leftarrow \Pi_{\mathsf{cc}}.\mathsf{Prove}(\mathsf{pk}, \mathbf{x}; \mathbf{w}, \tilde{o})$: The witness is partitioned as $\mathbf{w} := (\mathbf{u}, \omega)$ into a committed part and an uncommitted part. The prover outputs a proof π_{cc} together with a proof-dependent commitment $\tilde{\mathsf{cm}} = \mathsf{Com}.\mathsf{Commit}(\mathsf{ck}, \mathbf{u}; \tilde{o})$.
- $0/1 \leftarrow \Pi_{\mathsf{cc}}.\mathsf{Verify}(\mathsf{vk}, \mathbf{x}, \tilde{\mathsf{cm}}, \pi_{\mathsf{cc}})$: On input the verification key vk, a statement $\mathbf{x}$, a proof-dependent commitment $\tilde{\mathsf{cm}}$, and a proof π_{cc}, the verifier outputs 0(reject) or 1(accept)

3.5 Freivalds' Algorithm

Freivalds' algorithm [11, 15] is a simple yet powerful randomized method for verifying a matrix product without recomputing it. Let $\mathbf{X}, \mathbf{Y}, \mathbf{Z} \in \mathbb{F}^{n \times n}$ and consider the claim $\mathbf{X} \cdot \mathbf{Y} = \mathbf{Z}$. Instead of computing $\mathbf{XY}$ in $O(n^3)$ time, Freivalds samples a uniformly random vector $\mathbf{r} \xleftarrow{\$} \mathbb{F}^n$ and checks

$$\mathbf{X}(\mathbf{Yr}) \stackrel{?}{=} \mathbf{Zr}.$$

This verification runs in $O(n^2)$ field operations. This algorithm has soundness error: if $\mathbf{XY} = \mathbf{Z}$, it always accepts. Otherwise, when $\mathbf{XY} \neq \mathbf{Z}$, the check passes with probability at most $1/|\mathbb{F}|$, i.e.,

$$\Pr\left[\, \mathbf{X}(\mathbf{Yr}) = \mathbf{Zr} \;\middle|\; \mathbf{XY} \neq \mathbf{Z} \,\right] \leq \frac{1}{|\mathbb{F}|}.$$

The same principle extends to rectangular matrices and to the transposed (left-multiplied) variant $\mathbf{r}^\top\mathbf{XY} \stackrel{?}{=} \mathbf{r}^\top\mathbf{Z}$, which we use in Section 5 to verify products of varying dimensions.

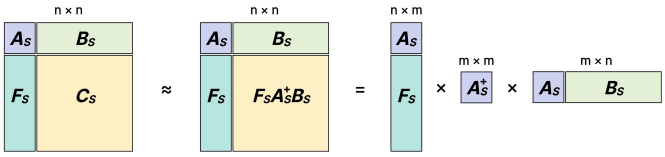
4 Technical Overview

The main computational bottleneck in Transformers is the quadratic (or higher) cost of the self-attention matrix multiplications. To address this, recent work [1, 7, 25, 28, 29] proposes alternatives that reduce the cost of self-attention, with several methods achieving near-linear complexity while maintaining accuracy. One important line of research is motivated by the observation that the self-attention matrix is approximately low rank. By projecting the input sequence into a lower-dimensional subspace, these methods retain the essential information while reducing the computational cost to $O(n)$. In the domain of zero-knowledge proofs, Freivalds' algorithm has been widely used to reduce the verification cost of matrix multiplication from $O(n^3)$ to $O(n^2)$. However, prior work has rarely explored how the inherent low-rank structure of self-attention can be exploited to further reduce the proof cost in zero-knowledge settings.

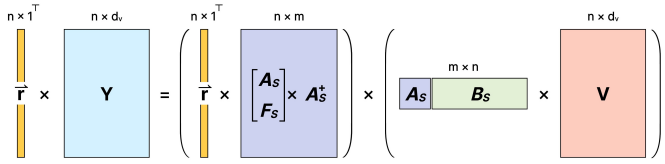
In this work, we propose **zkDietFormer**, which leverages the low-rank structure of self-attention to reduce proof cost to linear complexity. Our key idea is to avoid proving the full attention matrix and instead construct a zero-knowledge proof over its low-rank approximation. Since the choice of approximation determines what the circuit must prove, we first review the representative low-rank attention mechanisms proposed in the deep learning literature:

- Linformer [25] is based on the observation that the self-attention matrix is low rank. It projects the keys and values into a lower-dimensional space using learned projection matrices, thereby achieving linear complexity.
- Nyströmformer [29] applies the Nyström method to select representative landmark points from the full matrix and reconstruct the attention matrix from these landmarks.
- A^3 [28] analytically decomposes the attention computation into low-rank components without requiring additional training, deriving the approximation from the existing weight matrices.

We also considered Performer [7], which achieves linear complexity through kernelization rather than low-rank approximation. We do not adopt Performer because its FAVOR+ random feature map introduces additional nonlinear operations, requiring exponential evaluations for each random feature. In our design, we adopt Nyströmformer, as it does not require introducing additional learned projection matrices into the proving circuit, thereby simplifying the proof structure. Moreover, the theoretically grounded Nyström approximation provides numerically stable reconstruction regardless of sequence length.



(a) Nyströmformer matrix approximation



(b) Our zkDietFormer Freivalds' Check

Figure 1: Low-rank matrix multiplication with Freivalds' algorithm

As illustrated in Figure 1a, rather than directly handling the $n \times n$ self-attention matrix, we follow the Nyströmformer procedure and decompose it into a sequence of low-dimensional matrices of shapes $n \times m$, $m \times m$, and $m \times n$. we denote the three low-rank factors by $\tilde{F} := \text{softmax}(Q\tilde{K}^\top/\sqrt{d_k}) \in \mathbb{F}^{n\times m}$, $\tilde{A} := \text{softmax}(\tilde{Q}\tilde{K}^\top/\sqrt{d_k}) \in \mathbb{F}^{m\times m}$, and $\tilde{B} := \text{softmax}(\tilde{Q}K^\top/\sqrt{d_k}) \in \mathbb{F}^{m\times n}$, treated formally in Section 5.2. In the block decomposition of Figure 1a, $\tilde{F}$, $\tilde{A}^+$, and $\tilde{B}$ take the roles of $\begin{bmatrix} A_S \\ F_S \end{bmatrix}$, A_S^+, and $\begin{bmatrix} A_S & B_S \end{bmatrix}$, respectively; they are computed from the landmarks before the softmax rather than extracted from S (cf. Section 3.3), so Figure 1b is obtained from Figure 1a by this substitution. By applying Freivalds' algorithm to this low-rank representation (Figure 1b), we compress the verification workload from $O(n^2)$ to $O(n)$, effectively reducing it to a chain of vector–matrix products of total size $2nm + m^2 + nd_v$: multiplying the random vector by $\tilde{F}$ (nm), then by $\tilde{A}^+$ (m^2), then by $\tilde{B}$ (mn), and finally by $\mathbf{V}$ (nd_v). The pseudoinverse factor is instantiated by the iterative approximation Z^* of Section 3.3: the prover computes Z^* outside the circuit and supplies it as a witness, and our protocol verifies in-circuit that Z^* satisfies the Moore–Penrose conditions (Section 5.3) up to quantization tolerance. Since these conditions uniquely characterize the pseudoinverse $\tilde{A}^+$, this check constrains Z^* to a small neighborhood of $\tilde{A}^+$, without ever computing $\tilde{A}^+$ itself.

To verify such low-rank matrix multiplications using Freivalds' algorithm, we adopt a commit-carrying SNARK (CC-SNARK) architecture (Figure 2). To ensure that a fixed model is persistently used across multiple user queries, the model weight matrices W_Q, W_K, W_V, W_O are initially committed using a relation-independent key to establish a published global weight commitment (c_W). During execution, the CC-SNARK circuit extracts its own relation-dependent weight commitments (cm_W^i), and the prover demonstrates via CP_{link} that these internal commitments correspond to the same weights in c_W, while simultaneously proving the matrix multiplication result via Freivalds' algorithm with linear complexity. Instead of transmitting the full weight matrices, only the commitments are provided to the verifier, who then performs merely constant-time commitment checks. This design substantially reduces both proving and verification costs for matrix operations arising in large-scale models. Because zero-knowledge circuits operate over finite fields, real-valued Transformer computations cannot be represented directly. We therefore quantize the computations in a manner that minimizes precision loss, leveraging the R1CS-friendly quantization technique proposed in [10]. From this point onward, all matrices are understood to be over the finite field $\mathbb{F}$ after quantization.

5 zkDietFormer: Efficient Proofs of Low-Rank Matrix Multiplication via Freivalds' Algorithm

In this section, we present the core protocol of **zkDietFormer**. Our goal is to prove the correctness of Nyström -approximated low-rank self-attention while avoiding any explicit computation of the $n \times n$ attention matrix inside the circuit. Concretely, we design the protocol so that the number of constraints scales linearly with the sequence length n. To this end, we adopt a commit-carrying SNARK (CC-SNARK) architecture that pushes computationally expensive operations (e.g., large matrix multiplications and pseudoinverse computation) outside the circuit, and proves only their correctness inside the circuit using a small number of constraints.

5.1 Protocol Overview

Our protocol follows the message flow shown in Figure 3. Prior to answering any query, the model provider generates a long-lived global commitment $c_W \leftarrow \text{Com.Commit}(\text{ck}_{\text{glob}}, W; o)$ to its weight matrices $W = (W^Q, W^K, W^V, W^O)$ under randomness o using the

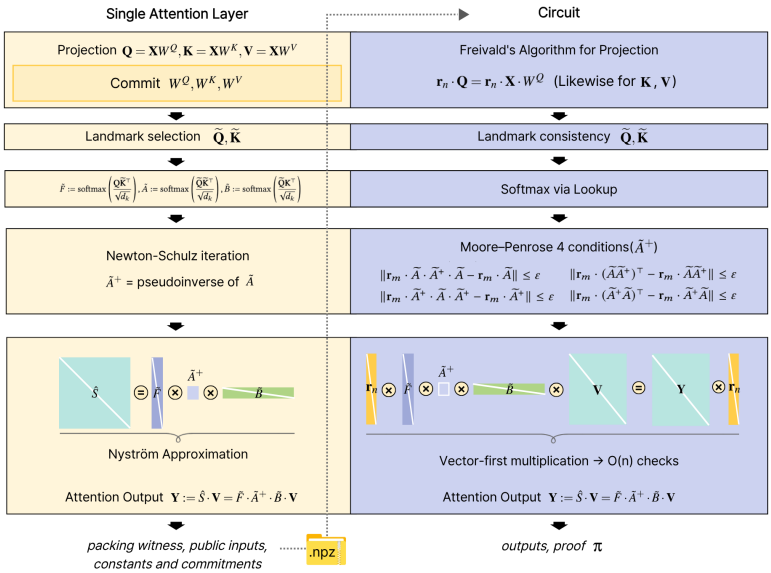


Figure 2: Overview of Self-Attention and our framework

Protocol

$\mathcal{P}(\mathsf{ck}, c_W, \mathbf{X}; \{W^Q, W^K, W^V, W^O\})$ $\qquad$ $\mathcal{V}(\mathsf{vk}, c_W, \mathbf{X})$

$(\mathbf{Y}', aux, \mathsf{lk}) \leftarrow \mathsf{NyströmAttention}(\mathbf{X}, W^Q, W^K, W^V, W^O)$

$\widetilde{\mathsf{cm}}_{W^i} \leftarrow \mathsf{Com.Commit}(\mathsf{ck}, W^i; \tilde{o}_i)$ for $i \in \{Q, K, V, O\}$

$\widetilde{\mathsf{cm}} \leftarrow \mathsf{Com.Commit}(\mathsf{ck}, \mathsf{lk}; \tilde{o})$

$\{\widetilde{\mathsf{cm}}_{W^i}\}_{i\in\{Q,K,V,O\}}, \widetilde{\mathsf{cm}}, \mathbf{Y}'$ $\longrightarrow$

$\mathbf{r}_n, \mathbf{r}_m$ $\longleftarrow$ $\qquad$ $\mathbf{r}_n \xleftarrow{\$} \mathbb{F}^n, \quad \mathbf{r}_m \xleftarrow{\$} \mathbb{F}^m$

$\mathbf{x} := \{\mathbf{X}, \mathbf{Y}', \mathbf{r}_n, \mathbf{r}_m\}$

$\mathbf{u} := \{(W^i, \tilde{o}_i)_{i\in\{Q,K,V,O\}}, (\mathsf{lk}, \tilde{o}), (W^i, o_i)_{i\in\{Q,K,V,O\}}\}$

$\omega := aux$

$\pi \leftarrow \Pi_{\mathsf{zkdiet}}.\mathsf{Prove}(\mathsf{pk}, \mathbf{x}; (\mathbf{u}, \omega))$

π $\longrightarrow$

$\Pi_{\mathsf{zkdiet}}.\mathsf{Verify}(\mathsf{vk}, \mathbf{x}, \{\widetilde{\mathsf{cm}}_{W^i}\}_{i\in\{Q,K,V,O\}}, \widetilde{\mathsf{cm}}, c_W, \pi)$

Figure 3: Our protocol

circuit-independent key $\mathsf{ck}_{\mathsf{glob}}$, and publishes c_W once to bind all future inferences to this specific model.

When a query arrives, the prover $\mathcal{P}$ first executes NyströmAttention locally outside the circuit on the input sequence $\mathbf{X} \in \mathbb{F}^{n\times d}$ and the weight matrices W. From this local evaluation, the prover obtains the final attention output $\mathbf{Y}'$, the non-linear intermediate operands $\mathsf{lk} := (\tilde{F}, \tilde{A}, \tilde{B})$, and the auxiliary intermediate matrices aux.

Next, the prover commits to the weight matrices W^Q, W^K, W^V, W^O under the circuit-dependent key ck, producing $\widetilde{\mathsf{cm}}_{W^i} = \mathsf{Com.Commit}(\mathsf{ck}, W^i; \tilde{o}_i)$ for $i \in \{Q, K, V, O\}$. The prover also commits to the non-linear operands lk, yielding the proof-dependent commitment $\widetilde{\mathsf{cm}} = \mathsf{Com.Commit}(\mathsf{ck}, \mathsf{lk}; \tilde{o})$.

The prover then transmits the weight commitments $\{\widetilde{\mathsf{cm}}_{W^i}\}_{i\in\{Q,K,V,O\}}$, the commitment $\widetilde{\mathsf{cm}}$, and the plaintext output $\mathbf{Y}'$ to the verifier $\mathcal{V}$. In response, the verifier samples random challenge vectors $\mathbf{r}_n \xleftarrow{\$} \mathbb{F}^n$ and $\mathbf{r}_m \xleftarrow{\$} \mathbb{F}^m$ for Freivalds' randomized checks and sends them back to the prover.

The public instance for the proof system is established as $\mathbf{x} := (\mathbf{X}, \mathbf{Y}', \mathbf{r}_n, \mathbf{r}_m)$.

The witness is organized into two primary categories: committed witnesses ($\mathbf{u}$) and uncommitted witnesses (ω). The committed part,

$$\mathbf{u} := \{(W^i, \tilde{o}_i)_{i\in\{Q,K,V,O\}}, (\mathsf{lk}, \tilde{o}), (W^i, o_i)_{i\in\{Q,K,V,O\}}\},$$

is bound by the local weight commitments $\{\widetilde{\mathsf{cm}}_{W^i}\}$ and the non-linear commitment $\widetilde{\mathsf{cm}}$, alongside the randomness o of the global commitment c_W. The uncommitted witness consists of the remaining intermediate matrices, $\omega := aux$, where the full auxiliary tuple is explicitly defined as

$$aux := \big(\mathbf{Q}, \mathbf{K}, \mathbf{V},\ \widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}},\ \mathbf{Q}\widetilde{\mathbf{K}}^\top,\ \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top,\ \widetilde{\mathbf{Q}}\mathbf{K}^\top,\ \tilde{F}, \tilde{A}, \tilde{B},\ \tilde{A}^+,\ \mathbf{Y}\big).$$

Although these witness intermediates carry no direct commitment, they cannot be manipulated by a malicious prover: each intermediate matrix is strictly constrained by the chain of circuit equalities (and Freivalds' checks) that bridge the committed weights on one end, the committed non-linear operands lk in the middle, and the public output $\mathbf{Y}'$ on the other. Because all committed values—both weight commitments and $\widetilde{\mathsf{cm}}$—are fixed prior to sampling the challenges $\mathbf{r}_n, \mathbf{r}_m$, the soundness of the Freivalds randomized verification is preserved.

For notational simplicity, we write $\tilde{A}^+$ for the corresponding pseudoinverse of $\tilde{A}$ throughout this section, with the understanding that the value actually supplied is the iterative approximation Z^* of Section 3.3, and that the Moore–Penrose checks of Section 5.3, verified up to quantization tolerance, are what license this identification.

Finally, the prover invokes the unified proof system to generate a composite proof $\pi \leftarrow \Pi_{\mathsf{zkdiet}}.\mathsf{Prove}(\mathsf{pk}, \mathbf{x}; (\mathbf{u}, \omega))$ and transmits π to the verifier. This unified proof π encapsulates the commit-carrying proof alongside two CP-linkages that bind $\{\widetilde{\mathsf{cm}}_{W^i}\}$ to the long-lived global commitment c_W and $\widetilde{\mathsf{cm}}$ to the lookup arguments. The verifier then checks the proof via $\Pi_{\mathsf{zkdiet}}.\mathsf{Verify}(\mathsf{vk}, \mathbf{x}, \{\widetilde{\mathsf{cm}}_{W^i}\}_{i\in\{Q,K,V,O\}}, \widetilde{\mathsf{cm}}, c_W, \pi)$. In the non-interactive variant, the challenges $\mathbf{r}_n, \mathbf{r}_m$ are derived via the Fiat–Shamir transformation from the transcript $(\mathbf{X}, \mathbf{Y}', \{\widetilde{\mathsf{cm}}_{W^i}\}_{i\in\{Q,K,V,O\}}, \widetilde{\mathsf{cm}}, c_W)$, ensuring that all commitments are bound before challenge generation.

5.2 Nyström -Based Low-Rank Self-Attention

Our system adopts a Nyström -based low-rank approximation for self-attention. In Nyströmformer , the query and key matrices are represented using m landmark vectors. Algorithm 1 describes the landmark selection procedure. Given $\mathbf{Q} = [\mathbf{q}_1, \ldots, \mathbf{q}_n]^\top$ and $\mathbf{K} = [\mathbf{k}_1, \ldots, \mathbf{k}_n]^\top$, we summarize the length-n sequence using m landmarks by partitioning it into m consecutive segments of length $\ell = n/m$ and defining each landmark as the average over the ℓ tokens in its segment. This yields $\widetilde{\mathbf{Q}} = [\tilde{\mathbf{q}}_1, \ldots, \tilde{\mathbf{q}}_m]^\top$ and $\widetilde{\mathbf{K}} = [\tilde{\mathbf{k}}_1, \ldots, \tilde{\mathbf{k}}_m]^\top$. With these landmarks, we avoid constructing the full $n\times n$ attention matrix and instead decompose the computation into three lower-dimensional matrices: $\mathbf{Q}\widetilde{\mathbf{K}}^\top \in \mathbb{F}^{n\times m}, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top \in \mathbb{F}^{m\times m}, \widetilde{\mathbf{Q}}\mathbf{K}^\top \in \mathbb{F}^{m\times n}$

We then apply the row-wise softmax to each of these matrices and compute

$$\tilde{F} := \mathrm{softmax}\Big(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\Big), \tilde{A} := \mathrm{softmax}\Big(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\Big), \tilde{B} := \mathrm{softmax}\Big(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\Big).$$

Algorithm 1 Select landmarks

Input: Query $\mathbf{Q}$, Key $\mathbf{K} \in \mathbb{F}^{n\times d}$; number of landmarks m with $m \mid n$
Output: Landmark matrices $\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}} \in \mathbb{F}^{m\times d}$

SelectLandmarks$(\mathbf{Q}, \mathbf{K}, m)$:

parse $\mathbf{Q} = [\mathbf{q}_1; \ldots; \mathbf{q}_n]^\top$, $\mathbf{K} = [\mathbf{k}_1; \ldots; \mathbf{k}_n]^\top$ (rows $\mathbf{q}_i, \mathbf{k}_i \in \mathbb{F}^d$)
$\ell \leftarrow n/m$
for $j \in \{1, \ldots, m\}$ **do**
  $\tilde{\mathbf{q}}_j \leftarrow \frac{1}{\ell}\sum_{i=(j-1)\ell+1}^{j\ell} \mathbf{q}_i, \quad \tilde{\mathbf{k}}_j \leftarrow \frac{1}{\ell}\sum_{i=(j-1)\ell+1}^{j\ell} \mathbf{k}_i$
endfor
$\widetilde{\mathbf{Q}} \leftarrow [\tilde{\mathbf{q}}_1; \ldots; \tilde{\mathbf{q}}_m]^\top$, $\widetilde{\mathbf{K}} \leftarrow [\tilde{\mathbf{k}}_1; \ldots; \tilde{\mathbf{k}}_m]^\top$
return $(\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}})$

Algorithm 2 Compute Moore–Penrose inverse

Input: selected landmark matrix $\tilde{A} \in \mathbb{F}^{m\times m}$
Output: approximate pseudoinverse $\tilde{A}^+ \in \mathbb{F}^{m\times m}$

ComputeMPInverse$(\tilde{A})$:

$Z_0 \leftarrow \tilde{A}^\top / \big(\|\tilde{A}\|_1 \cdot \|\tilde{A}\|_\infty\big)$
Set T (e.g., $T = 6$ iterations)
for $t \in \{0, \ldots, T-1\}$ **do**
  $Z_{t+1} \leftarrow \frac{1}{4} Z_t \big(13I - \tilde{A}Z_t\big(15I - \tilde{A}Z_t(7I - \tilde{A}Z_t)\big)\big)$
endfor
$\tilde{A}^+ \leftarrow Z_T$
return $\tilde{A}^+$

Here, $\tilde{A}$ is a small $m \times m$ matrix corresponding to the landmark submatrix of the original $n \times n$ attention matrix. Overall, the combination of $\tilde{F}$, $\tilde{A}$, and $\tilde{B}$ yields a Nyström approximation of the full attention map.

Finally, we compute an approximation to the Moore–Penrose pseudoinverse $\tilde{A}^+$ of the $m \times m$ matrix $\tilde{A}$. While one could compute $\tilde{A}^+$ via SVD, this is typically inefficient on GPU hardware. Therefore, Nyströmformer approximates $\tilde{A}^+$ via an iterative polynomial approximation to the SVD. Algorithm 2 shows this iterative procedure.

Using the resulting $\tilde{A}^+$, we form the final approximated attention matrix

$$\hat{S} := \tilde{F} \cdot \tilde{A}^+ \cdot \tilde{B},$$

and compute the attention output as

$$\mathbf{Y} := \hat{S} \cdot \mathbf{V}.$$

Finally, the output projection is applied using the weight matrix $\mathbf{W}^O \in \mathbb{F}^{d\times d}$:

$$\mathbf{Y}' := \mathbf{Y} \cdot \mathbf{W}^O.$$

Algorithm 3 provides the complete Nyström -based self-attention computation used in our framework.

Algorithm 3 Nyström approximation of attention matrix

Input: Input sequence $\mathbf{X} \in \mathbb{F}^{n \times d}$,
Model parameters $W^Q, W^K, W^V, W^O \in \mathbb{F}^{d \times d}$

Output: Attention output $\mathbf{Y}' \in \mathbb{F}^{n \times d}$,
Auxiliary intermediates aux,
Non-linear operands $\mathsf{lk} := (\tilde{F}, \tilde{A}, \tilde{B})$

$\mathsf{NyströmAttention}(\mathbf{X}, W^Q, W^K, W^V, W^O)$:

$\mathbf{Q} := \mathbf{X}W^Q, \quad \mathbf{K} := \mathbf{X}W^K, \quad \mathbf{V} := \mathbf{X}W^V$

$\widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}} \leftarrow \mathsf{SelectLandmarks}(\mathbf{Q}, \mathbf{K})$

$\tilde{F} := \mathrm{softmax}\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right), \quad \tilde{A} := \mathrm{softmax}\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right), \quad \tilde{B} := \mathrm{softmax}\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\right)$

$\tilde{A}^+ \leftarrow \mathsf{ComputeMPInverse}(\tilde{A})$

$\mathbf{Y} := (\tilde{F} \cdot \tilde{A}^+)(\tilde{B} \cdot \mathbf{V})$

$\mathbf{Y}' := \mathbf{Y} \cdot W^O$

$aux := (\mathbf{Q}, \mathbf{K}, \mathbf{V}, \widetilde{\mathbf{Q}}, \widetilde{\mathbf{K}}, \mathbf{Q}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\mathbf{K}^\top, \tilde{A}^+, \mathbf{Y})$

$\mathsf{lk} := (\tilde{F}, \tilde{A}, \tilde{B})$

return $(\mathbf{Y}', aux, \mathsf{lk})$

5.3 Efficient Verification via Freivalds' Algorithm

Our protocol uses Freivalds' algorithm as a core tool to efficiently verify that the low-rank self-attention computation is performed correctly. In particular, within the zk-SNARK circuit, we employ Freivalds' algorithm to ensure that the following key operations can be verified with linear cost.

Query/Key/Value matrix generation and output projection. Using the verifier's random vector $\mathbf{r}_n$, we check the relation $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q$ by comparing $\mathbf{r}_n^\top \mathbf{Q}$ with $\mathbf{r}_n^\top(\mathbf{X}\mathbf{W}^Q)$. We verify $\mathbf{K}$ and $\mathbf{V}$ in the same way. Rather than proving the full $n \times d$ matrix product directly, this Freivalds' check reduces verification to computing only the vector-matrix product $\mathbf{r}_n^\top \mathbf{X} W^Q$, thereby reducing the complexity from $O(nd^2)$ to $O(nd + d^2)$. The output projection $\mathbf{Y}' = \mathbf{Y}\mathbf{W}^O$ is verified analogously: we check $\mathbf{r}_n^\top \mathbf{Y}' \stackrel{?}{=} (\mathbf{r}_n^\top \mathbf{Y})\, W^O$.

Low-rank matrix operations. We also verify low-dimensional products such as $\mathbf{Q}\widetilde{\mathbf{K}}^\top$, $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top$, and $\widetilde{\mathbf{Q}}\mathbf{K}^\top$ by multiplying each matrix with a random vector of matching dimension, i.e., $\mathbf{r}_n$ or $\mathbf{r}_m$. For instance, we can certify the correctness of $\mathbf{Q}\widetilde{\mathbf{K}}^\top$ by checking the following equality:

$$\mathbf{r}_n^\top(\mathbf{Q}\widetilde{\mathbf{K}}^\top) \stackrel{?}{=} (\mathbf{r}_n^\top \mathbf{Q})\, \widetilde{\mathbf{K}}^\top$$

Similarly, the products $\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top$ and $\widetilde{\mathbf{Q}}\mathbf{K}^\top$ are validated with linear cost, without explicitly computing the full matrix multiplications inside the circuit.

Moore–Penrose pseudoinverse. As shown in Algorithm 2,computing the Moore–Penrose pseudoinverse $\tilde{A}^+$ inside the circuit is inefficient. Instead, the circuit certifies that the supplied matrix $\tilde{A}^+$ satisfies the four Moore–Penrose conditions, which correspond to lines 19–22 in Figure 4. Since $\tilde{A}^+$ is obtained by a finite Newton–Schulz iteration, these conditions are enforced up to a bounded residual ε; we defer the concrete tolerance to Section 7.1.

- $\|\tilde{A} \cdot \tilde{A}^+ \cdot \tilde{A} - \tilde{A}\| \le \varepsilon$
- $\|\tilde{A}^+ \cdot \tilde{A} \cdot \tilde{A}^+ - \tilde{A}^+\| \le \varepsilon$
- $\|(\tilde{A}\tilde{A}^+)^\top - \tilde{A}\tilde{A}^+\| \le \varepsilon$
- $\|(\tilde{A}^+\tilde{A})^\top - \tilde{A}^+\tilde{A}\| \le \varepsilon$

This allows the prover to efficiently demonstrate that $\tilde{A}^+$ approximately satisfies the pseudoinverse conditions without explicitly computing it in-circuit.

The relations that the prover must satisfy are formalized in Figure 4 as the definition of the circuit relation. In particular, the relation $\mathcal{R}(\mathbf{x}, \mathbf{w})$ takes the public instance $\mathbf{x} = (\mathbf{X}, \mathbf{r}_n, \mathbf{r}_m)$ and the witness $\mathbf{w} = (W^Q, W^K, W^V, W^O, \mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{Q}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top, \widetilde{\mathbf{Q}}\mathbf{K}^\top, \tilde{A}^+, \mathbf{Y}, \mathbf{O})$, and enforces all required equalities as well as the softmax computation. All key equalities are expressed in the form of products with the random vectors, accordingly, the circuit computes only vector inner products.

Relation

1: $\mathcal{R}(\mathbf{x}, \mathbf{w})$:
2: **parse** $\mathbf{x} = (\mathbf{X}, \mathbf{r}_n, \mathbf{r}_m, \mathbf{Y}'), \mathbf{w} = (\mathbf{u}, \omega)$
3: **parse** $\mathbf{u} := \{(W^i, \tilde{o}_i)_{i \in \{Q,K,V,O\}}, (\mathsf{lk}, \tilde{o}), (W^i, o_i)_{i \in \{Q,K,V,O\}}\}$
4: **parse** $\omega = aux$
5: **parse** $\mathbf{Q} = [\mathbf{q}_1, \mathbf{q}_2, \ldots, \mathbf{q}_n]^\top, \mathbf{K} = [\mathbf{k}_1, \mathbf{k}_2, \ldots, \mathbf{k}_n]^\top$
6: $\mathbf{r}_n^\top \cdot \mathbf{Q} = \mathbf{r}_n^\top \cdot \mathbf{X} \cdot W^Q$
7: $\mathbf{r}_n^\top \cdot \mathbf{K} = \mathbf{r}_n^\top \cdot \mathbf{X} \cdot W^K$
8: $\mathbf{r}_n^\top \cdot \mathbf{V} = \mathbf{r}_n^\top \cdot \mathbf{X} \cdot W^V$
9: **for** $j \in \{1, \ldots, m\}$ **do**
10: $\tilde{\mathbf{q}}_j \leftarrow \sum_{i=(j-1)\times\ell+1}^{j\ell} \frac{\mathbf{q}_i}{\ell}, \tilde{\mathbf{k}}_j \leftarrow \sum_{i=(j-1)\times\ell+1}^{j\ell} \frac{\mathbf{k}_i}{\ell}$
11: **endfor**
12: $\widetilde{\mathbf{Q}} \leftarrow [\tilde{\mathbf{q}}_1, \ldots, \tilde{\mathbf{q}}_m]^\top, \widetilde{\mathbf{K}} \leftarrow [\tilde{\mathbf{k}}_1, \ldots, \tilde{\mathbf{k}}_m]^\top$
13: $\mathbf{r}_n^\top \cdot \mathbf{Q}\widetilde{\mathbf{K}}^\top = \mathbf{r}_n^\top \cdot \mathbf{Q} \cdot \widetilde{\mathbf{K}}^\top$
14: $\mathbf{r}_m^\top \cdot \widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top = \mathbf{r}_m^\top \cdot \widetilde{\mathbf{Q}} \cdot \widetilde{\mathbf{K}}^\top$
15: $\mathbf{r}_m^\top \cdot \widetilde{\mathbf{Q}}\mathbf{K}^\top = \mathbf{r}_m^\top \cdot \widetilde{\mathbf{Q}} \cdot \mathbf{K}^\top$
16: $\widetilde{F} = \mathrm{softmax}\left(\frac{\mathbf{Q}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)$
17: $\widetilde{A} = \mathrm{softmax}\left(\frac{\widetilde{\mathbf{Q}}\widetilde{\mathbf{K}}^\top}{\sqrt{d_k}}\right)$
18: $\widetilde{B} = \mathrm{softmax}\left(\frac{\widetilde{\mathbf{Q}}\mathbf{K}^\top}{\sqrt{d_k}}\right)$
19: $\|\mathbf{r}_m^\top \cdot \widetilde{A} \cdot \widetilde{A}^+ \cdot \widetilde{A} - \mathbf{r}_m^\top \cdot \widetilde{A}\| \le \varepsilon$
20: $\|\mathbf{r}_m^\top \cdot \widetilde{A}^+ \cdot \widetilde{A} \cdot \widetilde{A}^+ - \mathbf{r}_m^\top \cdot \widetilde{A}^+\| \le \varepsilon$
21: $\|\mathbf{r}_m^\top \cdot (\widetilde{A}\widetilde{A}^+)^\top - \mathbf{r}_m^\top \cdot \widetilde{A}\widetilde{A}^+\| \le \varepsilon$
22: $\|\mathbf{r}_m^\top \cdot (\widetilde{A}^+\widetilde{A})^\top - \mathbf{r}_m^\top \cdot \widetilde{A}^+\widetilde{A}\| \le \varepsilon$
23: $\mathbf{r}_n^\top \cdot \mathbf{Y} = \mathbf{r}_n^\top \cdot \widetilde{F} \cdot \widetilde{A}^+ \cdot \widetilde{B} \cdot \mathbf{V}$
24: $\mathbf{r}_n^\top \cdot \mathbf{Y}' = \mathbf{r}_n^\top \cdot \mathbf{Y} \cdot W^O$

Figure 4: Relation of zkDietFormer

6 Security

In this section, we formalize the security guarantees provided by **zkDietFormer**. We first outline the system and threat models (Section 6.1), present our security definition for zero-knowledge verifiable inference over a low-rank attention functionality (Section 6.2), and finally delineate the attack vectors that fall outside our scope (Section 6.2).

6.1 System and Threat Model

Our system involves two parties, a *model provider* and a *user*. The provider holds the trained per-layer weight matrices $(W^i)_{i\in\{Q,K,V,O\}}$ of a Nyström-based Transformer and exposes an inference interface: the user submits an input sequence $\mathbf{X} \in \mathbb{F}^{n\times d}$ and receives the corresponding output $\mathbf{Y}'$. The architectural parameters $\text{zkdiet} = (n, d, h, L, m, k_{\text{bits}})$, including the number of landmarks m and the quantization precision, are public and fixed by the circuit; only the weight values are private. Before serving any query, the provider publishes a single commitment c_W to its weights under a circuit independent key, so that every subsequent response, including responses produced under a circuit instantiated for a different sequence length, is bound to one and the same model.

We denote the resulting functionality by $\mathbf{Y}' \leftarrow \mathcal{F}^{\varepsilon}_{\text{zkdiet}}(W, \mathbf{X})$, where $\mathcal{F}^{\varepsilon}_{\text{zkdiet}}$ is the quantized Nyström attention pipeline of Figure 4, and ε is the residual tolerance admitted for the pseudoinverse factor.

A zero-knowledge verifiable low-rank attention inference (zkDIET) scheme is a tuple of algorithms $\text{zkDIET} = (\text{KeyGen}, \text{Setup}, \text{Commit}, \text{Prove}, \text{Verify})$. We separate key generation into two algorithms because the two commitment layers of our protocol serve different purposes: one binds the model across its entire lifetime, the other binds a single proof.

- $\text{ck}_{\text{glob}} \leftarrow \text{zkDIET.KeyGen}(1^\lambda, N)$: given the security parameter λ and an upper bound N on the number of field elements to be committed, output a *circuit independent* commitment key. This key is sampled once and is shared by every circuit configuration the provider may later deploy.
- $(\text{ck}, \text{pk}, \text{vk}) \leftarrow \text{zkDIET.Setup}(1^\lambda, \mathcal{R}_{\text{zkdiet}})$: given the relation induced by the architectural parameters zkdiet, output a *circuit dependent* commitment key ck together with a proving key and a verification key. This algorithm is run once per architecture, not per query.
- $c_W \leftarrow \text{zkDIET.Commit}(\text{ck}_{\text{glob}}, W; o)$: commit to the weight matrices $W = (W^i)_{i\in\{Q,K,V,O\}}$ under randomness o. The provider publishes c_W once, before answering any query, and reuses it across all subsequent queries and across all circuit configurations that share these weights.
- $(\mathbf{Y}', \pi) \leftarrow \text{zkDIET.Prove}(\text{pk}, \text{ck}, \text{ck}_{\text{glob}}, \mathbf{X}; W, o)$: evaluate the pipeline on $\mathbf{X}$ outside the circuit to obtain the output $\mathbf{Y}'$, the non-linear operands lk and the auxiliary intermediates aux; invoke the commit-carrying backend, which returns a proof π_{cc} together with the proof-dependent commitments $\widetilde{\text{cm}}_W$ and $\widetilde{\text{cm}}_{\text{lk}}$ generated under ck; and produce a link proof π_{link} asserting that $\widetilde{\text{cm}}_W$ and c_W open to the same weights, and that $\widetilde{\text{cm}}_{\text{lk}}$ and the operand commitment of the lookup argument open to the same lk. The output proof is $\pi := (\pi_{cc}, \widetilde{\text{cm}}_W, \widetilde{\text{cm}}_{\text{lk}}, \pi_{\text{link}})$.
- $\{0, 1\} \leftarrow \text{zkDIET.Verify}(\text{vk}, \mathbf{X}, \mathbf{Y}', c_W, \pi)$: accept only if π_{cc} verifies against the public instance, the link proof holds, and the lookup argument accepts.

Note that the verifier takes the published c_W, not a per-proof commitment, as its trust anchor for the model. This is what allows a user to compare two responses issued weeks apart, or issued by two circuits instantiated for different sequence lengths, and conclude that they came from the same weights. The commit-carrying backend alone cannot offer this guarantee, since its commitment key is derived from the relation and therefore differs between configurations; the link proof is what bridges the per-proof commitment to the long-lived one. Both commitments are computed by the prover as group operations outside the circuit, which is precisely the cost we wish to keep out of the constraint system.

Threat Model. The two parties mutually distrust each other. The provider is *malicious*: it may answer with the output of a cheaper or stale model, silently swap weights between queries or between circuit configurations, or fabricate intermediate values such as the pseudoinverse witness in order to make an incorrect output verify. The user is *honest but curious*: it follows the protocol but tries to learn the weights from everything it observes, namely the published commitment, the per-proof commitments, the challenge vectors, and the proof itself. Under this model we target two properties.

- *Inference integrity.* An accepted $\mathbf{Y}'$ is the result of evaluating, under the declared architecture, the very weights fixed by the published commitment c_W, and not an arbitrary value. Since c_W is issued once and is independent of the circuit, this binds every response the provider ever gives to a single model.
- *Model privacy in the proof.* The transcript $\left(c_W, \widetilde{\text{cm}}_W, \widetilde{\text{cm}}_{\text{lk}}, \pi_{cc}, \pi_{\text{link}}\right)$ leaks nothing about W beyond what $\mathbf{Y}'$ itself reveals.

6.2 Security Definition

We keep the notation of Section 6.1 throughout: $\mathcal{R}_{\text{zkdiet}}$ is the relation induced by the architectural parameters $\text{zkdiet} = (n, d, h, L, m, k_{\text{bits}})$, and $\mathcal{F}^{\varepsilon}_{\text{zkdiet}}$ is the quantized Nyström attention pipeline of Figure 4. Given the residual tolerance ε for the pseudoinverse factor, the functionality uniquely determines the output $\mathbf{Y}' = \mathcal{F}^{\varepsilon}_{\text{zkdiet}}(W, \mathbf{X})$ for any valid input $(W, \mathbf{X})$.

Definition 6.1 (Zero-knowledge verifiable low-rank attention inference). zkDIET is secure for $\mathcal{F}^{\varepsilon}_{\text{zkdiet}}$ if it satisfies the following three properties.

Completeness. For every $W \in \mathbb{F}^{d\times d}$ and every $\mathbf{X} \in \mathbb{F}^{n\times d}$, for $\text{ck}_{\text{glob}} \leftarrow \text{KeyGen}(1^\lambda, N)$, $(\text{ck}, \text{pk}, \text{vk}) \leftarrow \text{Setup}(1^\lambda, \mathcal{R}_{\text{zkdiet}})$, $c_W \leftarrow \text{Commit}(\text{ck}_{\text{glob}}, W; o)$ and $(\mathbf{Y}', \pi) \leftarrow \text{Prove}(\text{pk}, \text{ck}, \text{ck}_{\text{glob}}, \mathbf{X}; W, o)$,

$$\Pr\left[\text{Verify}(\text{vk}, \mathbf{X}, \mathbf{Y}', c_W, \pi) = 1\right] = 1.$$

Soundness. For any PPT adversary $\mathcal{A}$, there exists a PPT extractor $\mathcal{E}$ such that

$$\Pr\left[\begin{array}{c} \text{Verify}(\text{vk}, \mathbf{X}, \mathbf{Y}^*, c^*_W, \pi^*) = 1 \\ \wedge\ c^*_W = \text{Commit}(\text{ck}_{\text{glob}}, W^*; o^*) \\ \wedge\ \mathbf{Y}^* \notin \mathcal{F}^{\varepsilon}_{\text{zkdiet}}(W^*, \mathbf{X}) \end{array} \middle| \begin{array}{l} \text{ck}_{\text{glob}} \leftarrow \text{KeyGen}(1^\lambda, N) \\ (\text{ck}, \text{pk}, \text{vk}) \leftarrow \text{Setup}(1^\lambda, \mathcal{R}_{\text{zkdiet}}) \\ (c^*_W, \mathbf{X}, \mathbf{Y}^*, \pi^*) \leftarrow \mathcal{A}(\text{ck}_{\text{glob}}, \text{ck}, \text{pk}, \text{vk}) \\ (W^*, o^*) \leftarrow \mathcal{E}^{\mathcal{A}}(\text{trans}_{\mathcal{A}}) \end{array}\right]$$

The adversary is free to choose c^*_W and its underlying weights W^* itself, which models a provider that publishes a commitment to

a model of its own choosing; what it cannot do is answer with anything other than that committed model.

Zero-knowledge. Write $\mathsf{Eval}(W, \mathbf{X})$ for the deterministic off-circuit evaluation of Algorithm 3, which the honest prover performs as the first step of Prove; whenever Algorithm 2 meets the residual bound ε we have $\mathsf{Eval}(W, \mathbf{X}) \in \mathcal{F}^{\varepsilon}_{\mathsf{zkdiet}}(W, \mathbf{X})$. There exists a PPT simulator $\mathcal{S} = (\mathcal{S}_1, \mathcal{S}_2)$ such that for every $W = (W^i)_{i\in\{Q,K,V,O\}}$ and every PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, where $\mathcal{A}_1$ selects $\mathbf{X}$ only after seeing the keys and the published commitment, the two experiments below are computationally indistinguishable. Barred symbols denote values produced by the simulator.

$\mathsf{Real}_{\mathcal{A},W}(\lambda)$	$\mathsf{Ideal}_{\mathcal{A},\mathcal{S},W}(\lambda)$
$\mathsf{ck}_{\mathrm{glob}} \leftarrow \mathsf{KeyGen}(1^\lambda, N)$	$(\overline{\mathsf{ck}}_{\mathrm{glob}}, \overline{\mathsf{ck}}, \overline{\mathsf{pk}}, \overline{\mathsf{vk}}, \bar{c}_W, \mathsf{td}) \leftarrow \mathcal{S}_1(1^\lambda, N, \mathcal{R}_{\mathsf{zkdiet}})$
$(\mathsf{ck}, \mathsf{pk}, \mathsf{vk}) \leftarrow \mathsf{Setup}(1^\lambda, \mathcal{R}_{\mathsf{zkdiet}})$	
$o \xleftarrow{\$} \mathcal{O};\ \ c_W \leftarrow \mathsf{Commit}(\mathsf{ck}_{\mathrm{glob}}, W; o)$	
$(\mathbf{X}, st) \leftarrow \mathcal{A}_1(\mathsf{ck}_{\mathrm{glob}}, \mathsf{ck}, \mathsf{pk}, \mathsf{vk}, c_W)$	$(\mathbf{X}, st) \leftarrow \mathcal{A}_1(\overline{\mathsf{ck}}_{\mathrm{glob}}, \overline{\mathsf{ck}}, \overline{\mathsf{pk}}, \overline{\mathsf{vk}}, \bar{c}_W)$
$\mathbf{Y}' := \mathsf{Eval}(W, \mathbf{X})$	$\mathbf{Y}' := \mathsf{Eval}(W, \mathbf{X})$
$\pi \leftarrow \mathsf{Prove}(\mathsf{pk}, \mathsf{ck}, \mathsf{ck}_{\mathrm{glob}}, \mathbf{X}; W, o)$	$\overline{\pi} \leftarrow \mathcal{S}_2(\mathsf{td}, \mathbf{X}, \mathbf{Y}')$
return $\mathcal{A}_2(st, \mathbf{Y}', \pi)$	**return** $\mathcal{A}_2(st, \mathbf{Y}', \overline{\pi})$

$$\left|\Pr\left[\mathsf{Real}_{\mathcal{A},W}(\lambda) = 1\right] - \Pr\left[\mathsf{Ideal}_{\mathcal{A},\mathcal{S},W}(\lambda) = 1\right]\right| \le \mathsf{negl}(\lambda).$$

Both experiments hand $\mathcal{A}_2$ the same output $\mathbf{Y}'$, and the simulator receives only $(\mathbf{X}, \mathbf{Y}')$, never W and never the opening o. The definition therefore states that the entire transcript the user observes, namely c_W, the proof-dependent commitments $\widetilde{\mathsf{cm}}_W, \widetilde{\mathsf{cm}}_{lk}$ carried inside π, the challenge vectors, and the proof itself, reveals nothing about the weights beyond what $\mathbf{Y}'$ already reveals.

Out-of-Scope Attacks. While Definition 6.1 guarantees inference integrity and weight privacy within the proof system, several orthogonal threats fall outside the scope of this work. First, a user issuing many queries may attempt to reconstruct the model weights W from the returned outputs $\mathbf{Y}'$ alone, entirely without relying on the proof. Defenses against such black-box model extraction (e.g., rate limiting, output truncation, or watermarking) are studied independently and can be layered on top of our protocol.

Second, a commitment binds the provider to a single model, but does not guarantee model quality; an adversary could commit to a backdoored or low-accuracy model and continuously produce valid proofs for it. Attesting to the quality or fairness of the committed model requires auxiliary mechanisms (such as proving accuracy on a public labeled dataset), which we leave for future investigation.

7 Experiments

In this section, we evaluate the concrete proof generation costs of **zkDietFormer** and compare them against a standard self-attention baseline. Because our contribution targets the architectural cost of proving attention—i.e., the circuit structure rather than the learned weights—the evaluation focuses on the cost of the circuit itself. We use modified HuggingFace BERT and Nyströmformer configurations (with adjusted hidden dimensions and layer counts) to enable proving across multiple model sizes. Because these configurations are evaluated without per-spec fine-tuning, model inference accuracy is not the focus of this evaluation.

7.1 Implementation

We implement **zkDietFormer** in two parts: a Python-based witness generation pipeline and a Rust-based zero-knowledge proving circuit. Witness data (model weights and intermediate activations) are generated using PyTorch 3.11.15 and HuggingFace Transformers 5.8.0, which instrument the standard `BertForMaskedLM` and `NystromformerForMaskedLM` model classes with forward hooks to capture per-layer activations. The Rust circuit is built on the arkworks library over the BLS12-381 elliptic curve and uses the BLAKE2s hash function for cryptographic operations.

We use LegoGroth16 [4] as our commit-carrying SNARK (CC-SNARK) instantiation. Non-arithmetic operations—softmax exponentials, SigmoidGELU activation, and LayerNorm inverse square root—are evaluated using Lasso [21] lookup arguments.

Pseudoinverse Computation. As described in Section 5.3, the circuit verifies the prover-supplied $\tilde{A}^+$ against the Moore–Penrose conditions up to a bounded residual enforced via element-wise range checks. We empirically evaluated the maximum element-wise residual under a tiny-spec configuration (m=32 landmarks and 12-bit quantization). The observed maximum residuals for the verified conditions range from 1 to 161 relative to the full quantization range $[0, 4095]$. To accommodate these residuals, the circuit enforces bounds of $\epsilon \in \{4, 64, 512\}$, providing a 3× to 5× safety margin above the empirical maximum errors, which strictly remain below 4% of the full range.

Quantization. Zero-knowledge circuits operate over finite fields and cannot represent floating-point arithmetic directly. We therefore quantize all model computations following the R1CS-friendly quantization scheme of ZEN [10]. All weights and activations are converted to 12-bit asymmetric fixed-point representations, and the circuit precision parameter is set to $k_{\mathrm{bits}} = 26$, providing sufficient headroom for accumulated multiplication errors across a full layer. Different operators use different fractional precisions to maintain numerical fidelity: LayerNorm uses 8-bit fractional precision (`FRAC_BITS` = 8); Softmax uses 16-bit fractional precision (`FRAC_BITS` = 16), reflecting the higher dynamic range required by the exponential function; and SigmoidGELU is evaluated via lookup tables of size $2^{16} = 65{,}536$ entries, embedded into the circuit using lookup arguments.

Activation Function Replacement. While our primary focus is on the attention sub-circuit, we implemented and measured the full transformer pipeline (Attention + MLP + LayerNorm); detailed MLP + LayerNorm measurements are reported in Appendix A. One modification required by this end-to-end implementation concerns the GELU activation used in the MLP block. Both BERT and Nyströmformer use GELU [13] as their default activation, typically implemented via the approximation $\mathrm{GELU}(x) \approx 0.5\,x\left(1 + \tanh\left[\sqrt{2/\pi}\,(x + 0.044715\,x^3)\right]\right)$. We replace GELU in both models with SigmoidGELU, defined as

$$\mathrm{SigmoidGELU}(x) = x \cdot \sigma(1.702\,x),$$

where $\sigma(\cdot)$ denotes the logistic sigmoid. This alternative, originally noted by Hendrycks and Gimpel [13] as a close approximation to GELU, is substantially more circuit-friendly because the sigmoid function can be evaluated via a single lookup table, whereas the

tanh-based form requires a more involved computational procedure. The replacement is applied to all layers in both models.

7.2 Experimental Configuration

Hardware. All experiments are conducted on a Mac Studio equipped with an Apple M1 Ultra chip (20 cores: 16 performance + 4 efficiency) and 128 GB of unified memory, running macOS 14.6.1. No GPU acceleration is used, as the underlying ZK library operates exclusively on CPU. All reported timing measurements are CPU wall-clock time using Rayon's default thread pool, which utilizes all available CPU cores (20 cores on our platform). Peak memory is measured as the maximum resident set size (RSS) via the system `time` utility.

Baselines. We adopt two publicly available Transformer architectures from the HuggingFace Transformers library [27]:

- **BERT** (`bert-base-uncased`, Devlin et al. [9]): employs standard scaled dot-product self-attention with $O(n^2 d)$ complexity.
- **Nyströmformer** (`uw-madison/nystromformer-512`, Xiong et al. [29]): replaces the full attention matrix with a Nyström low-rank approximation, reducing attention complexity to $O(n)$.

Both models are instantiated using the standard HuggingFace classes (`BertForMaskedLM` and `NystromformerForMaskedLM`) without any custom architectural modifications beyond the activation replacement and quantization.

Model Configurations. We evaluate five configurations spanning the standard BERT model family, from the smallest (Tiny) to the full-scale (Base) model. The configurations are summarized in Table 1. Here, L denotes the number of Transformer layers, d the hidden dimension, and h the number of attention heads. For each model scale, the BERT baseline and Nyströmformer variant share identical structural parameters (L, d, h), differing only in the attention kernel.

Model	L	d	h
Tiny	2	128	2
Mini	4	256	4
Small	4	512	8
Medium	8	512	8
Base	12	768	12

Table 1: Model configurations across the standard BERT family (Tiny to Base)

7.3 Results

All reported metrics are *per single layer*: each table entry is the average of the per-layer measurements across all L layers of the model (e.g., L=2 for Tiny). Timing values (setup, commit, prove, verify) and proof size are averaged over the L layers. Memory denotes the peak resident set size (RSS) observed during a single process that proves all L layers sequentially; because it is a process-wide maximum, it is *not* averaged but reported as a single value. We measure six sequence lengths $n \in \{128, 256, 512, 1024, 2048, 4096\}$ with a fixed landmark count m=32 for Nyström attention. This range is chosen to expose the asymptotic advantage of Nyström attention over standard self-attention: since the landmark count m is held fixed while n varies, the Nyström approximation scales as $O(n)$ in both time and memory [29], whereas standard self-attention scales as $O(n^2)$. The cost gap between the two thus widens as n grows, and our sweep is designed to capture this transition—from short sequences where the two are comparable, to long sequences where the quadratic baseline becomes prohibitively expensive. For each circuit instance, we report constraint count, setup time, prove time, verify time, peak memory, and two commitment costs: the Groth16 committing time (G16) for committing to the circuit witness, and the Lasso committing time for committing to the lookup operands. Runs that exceed available memory are reported as out-of-memory (OOM); these patterns are themselves informative results, as they indicate the practical scalability boundary of each circuit variant. We evaluate **zkDietFormer**by applying its two constituent techniques—low-rank (Nyström) attention and Freivalds-based matrix-multiplication verification—one at a time, first isolating each technique's individual contribution and then combining both to measure the advantage of the full scheme. In our benchmarks, we pass only cryptographic hashes of the input sequence $\mathbf{X}$ and the attention output $\mathbf{Y}'$ as the public instance, while supplying $\mathbf{X}$ and $\mathbf{Y}'$ themselves as witnesses. This approach avoids the prohibitive setup cost incurred by large public-input vectors. Note that this is a deployment-time configuration; the protocol definition in Figure 3 presents $\mathbf{X}$ and $\mathbf{Y}'$ directly as public instances for clarity.

7.3.1 Low-Rank Attention is Essential for Asymptotic Scaling. To isolate the contribution of the attention kernel, we fix the verification method and compare the two kernels. The BERT-Naive and Nys-Naive rows of Table 2 hold verification at naive (in-circuit matrix multiplication) and compare BERT vs. Nyström attention on the Tiny configuration (L=2, d=128, h=2).

Under naive verification, the $O(n^2)$-versus-$O(n)$ gap is modest at short sequences but widens as n grows. At n=128 BERT-naive requires 15.06M per-layer constraints versus 13.47M for Nyström-naive (1.1×). By n=256 the ratio widens to 39.22M vs. 26.49M (1.5×), with per-layer setup time diverging from 911.89 s to 693.76 s (1.3×) and prove time from 337.23 s to 187.00 s (1.8×). Both BERT-naive and Nyström-naive exhaust our 128 GB memory at $n \geq 512$, highlighting the severity of the naive approach under the weight-commitment scheme. The only metric on which Nyström -naive is slower than BERT-naive is verify time (606.0 ms vs. 328.0 ms) at n=128. This residual gap reflects a constant overhead in our Nyström pipeline: the Nyström circuit must additionally verify four Moore–Penrose conditions and three separate row-wise softmax operations, each of which adds a fixed per-layer cost that is independent of n. Since both configurations use identical naive verification and differ only in their attention kernel, this widening gap suggests that the $O(n)$ versus $O(n^2)$ asymptotics of the two attention mechanisms translate into the asymptotics of their ZK proof generation cost.

7.3.2 Freivalds Removes the Constant Factor. We now hold the attention kernel fixed at Nyström and compare verification methods. The Nys-Naive and Nys-Freivalds rows of Table 2 compare two ways of proving the matrix multiplications inside Nyström attention: a *naive* implementation that re-derives each matrix product inside the circuit, the most direct way of arithmetizing matrix multiplication, and our *Freivalds-based* verification, in which the prover

Table 2: Full 2×2 factorial design—attention kernel (BERT / Nyström) × matrix-multiplication verification (Naive / Freivalds)—on the Tiny configuration (L=2, d=128, h=2). All metrics are per single layer (averaged over L layers). OOM entries (—) indicate that setup exhausted 128 GB; for these runs the Peak RAM column reports the observed process-wide maximum before termination.

				Commit (ms)					
n	Configuration	Constr. (M)	Setup (s)	G16	Lasso	Prove (s)	Verify (ms)	Proof Size (KB)	Peak RAM (GB)
128	BERT-Naive	15.06	362.83	1,129.5	159.5	81.21	328.0	900.0	73.04
	BERT-Freivalds	4.97	164.29	1,133.5	154.5	33.91	302.0	900.0	29.63
	Nys-Naive	13.47	343.81	1,115.5	265.0	81.20	606.0	1,777.3	67.22
	Nys-Freivalds	4.86	160.51	1,117.5	255.5	35.73	610.5	1,777.3	29.78
256	BERT-Naive	39.22	911.89	1,544.5	280.0	337.23	460.5	1,138.9	112.89
	BERT-Freivalds	10.72	407.99	1,410.5	307.5	85.80	392.0	1,138.9	67.24
	Nys-Naive	26.49	693.76	1,247.5	314.0	187.00	675.5	1,934.1	112.44
	Nys-Freivalds	9.55	318.63	1,219.5	296.5	71.92	653.0	1,934.1	54.78
512	BERT-Naive	—	—	—	—	—	—	—	>113 (OOM)
	BERT-Freivalds	24.79	917.80	2,550.0	672.5	216.22	618.0	1,620.5	112.38
	Nys-Naive	—	—	—	—	—	—	—	>113 (OOM)
	Nys-Freivalds	18.93	643.75	1,431.0	425.0	152.04	781.5	2,202.7	105.35
1024	BERT-Naive	—	—	—	—	—	—	—	>113 (OOM)
	BERT-Freivalds	—	—	—	—	—	—	—	>113 (OOM)
	Nys-Naive	—	—	—	—	—	—	—	>113 (OOM)
	Nys-Freivalds	37.69	1,405.98	2,319.0	592.5	629.29	893.0	2,492.4	112.42

commits the result outside the circuit and the circuit checks a randomized identity. For a product of two n×n matrices, this randomized check costs only $O(n^2)$ field operations—one order lower than the $O(n^3)$ cost of materializing the product inside the circuit. The two rows share the same low-rank attention structure; they differ only in how matrix multiplications are verified, isolating Freivalds' contribution.

Across all measured sequence lengths, Freivalds verification reduces the Nyström per-layer constraint count by a consistent factor of ∼2.8×. At n=256, for example, the per-layer constraint count drops from 26.49M (Naive) to 9.55M (Freivalds); per-layer setup time from 693.76 s to 318.63 s (2.2×); and prove time from 187.00 s to 71.92 s (2.6×). The reduction ratio remains stable across all sequence lengths, confirming that Freivalds provides a uniform constant-factor improvement independent of n. Without Freivalds, Nyström-naive exhausts our 128 GB memory at n≥512; with Freivalds the circuit extends to n=1024 (37.69M constraints, 629.29 s prove time).

7.3.3 Freivalds Alone Cannot Substitute for Low-Rank Attention. Freivalds is equally effective on standard BERT attention: comparing the BERT-Naive and BERT-Freivalds rows of Table 2, the per-layer constraint count drops from 15.06M to 4.97M at n=128 (3.0×) and from 39.22M to 10.72M at n=256 (3.7×), comparable to the ∼2.8× observed for Nyström . This confirms that Freivalds is a generic optimization rather than an artifact of Nyström structure. Yet even with Freivalds applied, BERT attention still exhausts our 128 GB memory at n≥1024 (Table 2), because the underlying $O(n^2)$ complexity is left intact. In contrast, Nyström-Freivalds remains feasible up to n=1024 with 37.69M constraints and 629.29 s prove time. These results suggest that the constant-factor reduction from Freivalds and the asymptotic reduction from low-rank attention provide complementary benefits, making their combination effective for improving scalability across a wide range of sequence lengths.

7.3.4 Combined Effect. Having decomposed each optimization axis, we now evaluate their combined effect across the two extremes of our design space: the conventional baseline (BERT-Naive) versus our full scheme (Nys-Freivalds). This diagonal contrast in Table 2 quantifies the total overall reduction achieved by integrating both optimizations.

At n=256, this combination reduces per-layer constraints from 39.22M (BERT-Naive) to 9.55M (Nyström-Freivalds)—a ∼4.1× reduction. The per-layer setup time shrinks from 911.89 s to 318.63 s (2.9×), and prove time drops from 337.23 s to 71.92 s (4.7×). Even at the shortest sequence (n=128), where both methods are feasible, the combined savings reach 3.1× in constraints, 2.3× in setup, and 2.3× in prove time. Peak memory drops from 73.04 GB to 29.78 GB (2.5×) at n=128 and from 112.89 GB to 54.78 GB (2.1×) at n=256. Crucially, BERT-Naive exhausts our 128 GB memory at n≥512, whereas Nyström-Freivalds remains feasible up to n=1024 (37.69M constraints, 629.29 s prove time). These joint savings show that combining low-rank attention (structural, $O(n)$ vs. $O(n^2)$) and Freivalds verification (constant-factor, ∼3×) makes it practical to generate zero-knowledge proofs of transformer attention even at sequence lengths that were difficult to reach under a limited memory budget. Full MLP + LayerNorm cost breakdowns are deferred to Appendix A.

7.3.5 Multi-Spec Scaling. Figure 5 extends the kernel comparison of Table 2 from the Tiny configuration to the full BERT model family, isolating the benefit of low-rank attention by holding the verification method fixed (Freivalds) and comparing BERT vs. Nyström attention.

The plot uses log-log scale and reports per-layer prove time (seconds) against sequence length n. Each line terminates at the largest n feasible within our 128 GB memory budget. Red lines correspond to BERT and blue lines to Nyström . Within each color family, lighter

shades represent smaller configurations and darker shades represent larger ones. Legend abbreviations: Bt/Ny = BERT/Nyström .

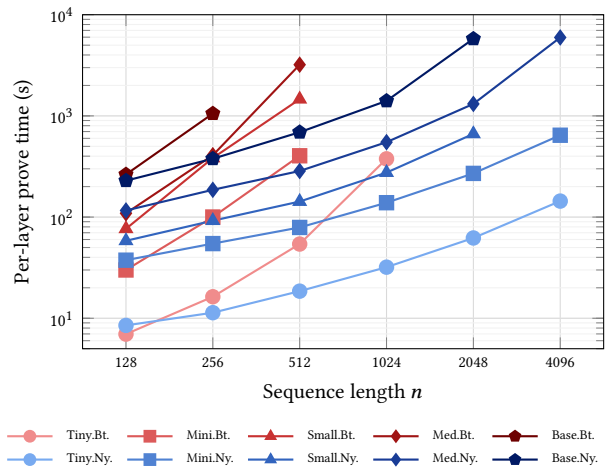


Figure 5: Effect of low-rank attention on per-layer prove time: BERT (red) vs. Nyström (blue) attention under Freivalds verification, as a function of sequence length n.

The most striking feature is the growth-rate gap: BERT's curves (red) rise noticeably faster than Nyström 's (blue), consistent with the $O(n^2)$ versus $O(n)$ asymptotic difference in attention complexity. This translates directly into memory exhaustion: at every configuration, BERT-Freivalds (red) exhausts our 128 GB memory budget at a substantially shorter n than Nyström -Freivalds (blue). The feasibility gap widens as d grows: at Base (d=768, darkest shades), BERT caps at n=256 while Nyström reaches n=2048, an 8× extension. Note that the feasibility limits in Figure 5 reflect a dedicated prove-time measurement run whose public-input overhead is lower than that of the full-metric runs reported in Table 2.

Quantization Fidelity.

Table 3: Quantization fidelity of 26-bit fixed-point arithmetic on a pretrained Nyströmformer (d=768, 12 layers, 12 heads, m=64 landmarks) at n=128.

Top-1 Agree. (%)		Top-5	Cosine	RMSE
All pos.	Mask pos.	All (%)	Sim.	(logit)
98.8	90.0	100.0	0.9975	0.378

Table 3 evaluates the impact of 26-bit fixed-point quantization (k_{bits}=26) on the pretrained uw-madison/nystromformer-512 (vocabulary size 30 000, pretrained on BookCorpus + English Wikipedia) at sequence length n=128 over 10 batches under full cascade evaluation, where quantization errors accumulate through all 12 attention, MLP, and LayerNorm layers. Note that the pretrained model uses m=64 landmarks (its default configuration), whereas the circuit experiments in Section 7.3 use m=32. Top-1 agreement over all positions remains 98.8% and top-5 agreement is 100%, with cosine similarity 0.9975. At the single `[MASK]` position the top-1 agreement drops to 90% but top-5 agreement stays at 100%, indicating that quantization primarily reshuffles near-tie predictions rather than introducing qualitatively wrong answers. These results confirm that 26-bit fixed-point arithmetic provides sufficient precision for faithful ZK-SNARK proving of transformer inference.

7.4 CONCLUSION AND FUTURE WORK

Practical impact. Our measurements across all five configurations (Figure 5) demonstrate that the combination of Nyström low-rank attention and Freivalds randomized matrix-multiplication verification yields multiplicative constraint savings that translate directly into longer provable sequences.

Memory as the practical bottleneck. Failed runs terminate during the Groth16 trusted setup phase, which materializes the full constraint system in RAM. Peak RSS for failed runs reaches ∼118–122 GB (the effective usable limit on our 128 GB machine), confirming that setup memory—not prover computation time—is the dominant scaling bottleneck.

The MLP frontier.. While our work removes attention as the scaling bottleneck, the feed-forward MLP block emerges as the next memory frontier. We measured the cost of proving the MLP subcircuit—identical between BERT and Nyströmformer and shared by both pipelines—and report the detailed results in Appendix A. The measurements confirm that, once attention is optimized, the MLP becomes the dominant constraint on the maximum provable sequence length: at d=512 and above, MLP exceeds 128 GB at all sequence lengths including n=128. Proving full-model transformer inference at sequence lengths beyond what this work demonstrates therefore requires either reducing the MLP's per-layer constraint footprint or infrastructure-level solutions such as streaming setup, distributed proving, or GPU-accelerated SNARK libraries. We view efficient ZK proving of the MLP block as a natural follow-up direction.

Per-layer cost is depth-invariant. A consistent finding across our measurements is that per-layer attention circuit cost depends only on hidden dimension d and sequence length n, not on model depth L. Small (L=4, d=512) and Medium (L=8, d=512) yield identical per-layer constraints, setup, and prove time; Medium's full-model costs are exactly 2× Small's. This depth-invariance implies that our results extrapolate to deeper architectures through simple multiplication by L: a 24-layer model with Base-scale dimensions would require ∼2× the prove time and setup we report for L=12. The fundamental constraint is per-layer memory, not depth.

Limitations. Our work has several limitations. *(i) Single-machine CPU-only environment.* All measurements use an Apple M1 Ultra without GPU acceleration. GPU-accelerated proving or distributed proving across multiple machines would shift relative costs, particularly for larger circuits. *(ii) Trusted setup.* Our Groth16-based system requires a circuit-specific trusted setup whose memory cost is our primary scaling bottleneck. Transparent SNARKs (e.g., FRI-based or IPA-based) would eliminate this at the cost of larger proof sizes and slower verification.*(iii) MLP sub-circuit.* The attention sub-circuit is made tractable by our techniques, but the MLP sub-circuit remains memory-bound at $d \geq 256$. Proving the full transformer (attention + MLP + LayerNorm) at the scales attempted here requires further techniques for the MLP block.

References

[1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. arXiv:2004.05150 [cs.CL] https://arxiv.org/abs/2004.05150

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. arXiv:2005.14165 [cs.CL] https://arxiv.org/abs/2005.14165

[3] Matteo Campanelli, Antonio Faonio, Dario Fiore, Anaïs Querol, and Hadrián Rodríguez. 2020. Lunar: a Toolbox for More Efficient Universal and Updatable zkSNARKs and Commit-and-Prove Extensions. Cryptology ePrint Archive, Paper 2020/1069. https://eprint.iacr.org/2020/1069

[4] Matteo Campanelli, Dario Fiore, and Anaïs Querol. 2019. LegoSNARK: Modular Design and Composition of Succinct Zero-Knowledge Proofs. Cryptology ePrint Archive, Paper 2019/142. https://doi.org/10.1145/3319535.3339820

[5] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems* (Athens, Greece) *(EuroSys '24)*. Association for Computing Machinery, New York, NY, USA, 560–574. https://doi.org/10.1145/3627703.3650088

[6] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating Long Sequences with Sparse Transformers. arXiv:1904.10509 [cs.LG] https://arxiv.org/abs/1904.10509

[7] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. 2022. Rethinking Attention with Performers. arXiv:2009.14794 [cs.LG] https://arxiv.org/abs/2009.14794

[8] Mingshu Cong, Tsz Hon Yuen, and Siu-Ming Yiu. 2024. zkMatrix: Batched Short Proof for Committed Matrix Multiplication. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security* (Singapore, Singapore) *(ASIA CCS '24)*. Association for Computing Machinery, New York, NY, USA, 289–305. https://doi.org/10.1145/3634737.3645003

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs.CL] https://arxiv.org/abs/1810.04805

[10] Boyuan Feng, Lianke Qin, Zhenfei Zhang, Yufei Ding, and Shumo Chu. 2021. ZEN: An Optimizing Compiler for Verifiable, Zero-Knowledge Neural Network Inferences. Cryptology ePrint Archive, Paper 2021/087. https://eprint.iacr.org/2021/087

[11] Rusins Freivalds. 1977. Probabilistic Machines Can Use Less Running Time. In *IFIP Congress*. https://api.semanticscholar.org/CorpusID:45405368

[12] Zahra Ghodsi, Tianyu Gu, and Siddharth Garg. 2017. SafetyNets: Verifiable Execution of Deep Neural Networks on an Untrusted Cloud. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2017/file/6048ff4e8cb07aa60b6777b6f7384d52-Paper.pdf

[13] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian Error Linear Units (GELUs). arXiv:1606.08415 [cs.LG] https://arxiv.org/abs/1606.08415

[14] Byeongjun Jang, Gweonho Jeong, Hyuktae Kwon, Hyunok Oh, and Jihye Kim. 2024. VECTIS: Efficient Batching Framework for Group-based CP-SNARKs. Cryptology ePrint Archive, Paper 2024/1405. https://eprint.iacr.org/2024/1405

[15] Ivan Korec and Jiří Wiedermann. 2014. Deterministic Verification of Integer Matrix Multiplication in Quadratic Time. In *SOFSEM 2014: Theory and Practice of Computer Science*, Viliam Geffert, Bart Preneel, Branislav Rovan, Július Štuller, and A. Min Tjoa (Eds.). Springer International Publishing, Cham, 375–382.

[16] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2024. vCNN: Verifiable Convolutional Neural Network Based on zk-SNARKs. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 4254–4270. https://doi.org/10.1109/TDSC.2023.3348760

[17] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) *(CCS '21)*. Association for Computing Machinery, New York, NY, USA, 2968–2985. https://doi.org/10.1145/3460120.3485379

[18] Meta. 2024. Llama 4 Model Card. GitHub repository documentation. https://github.com/meta-llama/llama-models/blob/main/models/llama4/MODEL_CARD.md Accessed: 2025-12-31.

[19] OpenAI. 2025. GPT-5 Model. OpenAI API Documentation. https://platform.openai.com/docs/models/gpt-5 Accessed: 2025-12-31.

[20] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. 2019. Compressive Transformers for Long-Range Sequence Modelling. arXiv:1911.05507 [cs.LG] https://arxiv.org/abs/1911.05507

[21] Srinath Setty, Justin Thaler, and Riad Wahby. 2023. Unlocking the lookup singularity with Lasso. Cryptology ePrint Archive, Paper 2023/1216. https://eprint.iacr.org/2023/1216

[22] Haochen Sun, Jason Li, and Hongyang Zhang. 2024. zkLLM: Zero Knowledge Proofs for Large Language Models. arXiv:2404.16109 [cs.LG] https://arxiv.org/abs/2404.16109

[23] Justin Thaler. 2013. Time-Optimal Interactive Proofs for Circuit Evaluation. In *Advances in Cryptology – CRYPTO 2013*, Ran Canetti and Juan A. Garay (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 71–89.

[24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. arXiv:1706.03762 [cs.CL] https://arxiv.org/abs/1706.03762

[25] Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. 2020. Linformer: Self-Attention with Linear Complexity. arXiv:2006.04768 [cs.LG] https://arxiv.org/abs/2006.04768

[26] Jiasi Weng, Jian Weng, Gui Tang, Anjia Yang, Ming Li, and Jia-Nan Liu. 2023. pvCNN: Privacy-Preserving and Verifiable Convolutional Neural Network Testing. arXiv:2201.09186 [cs.CR] https://arxiv.org/abs/2201.09186

[27] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander Rush. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 38–45.

[28] Jeffrey T. H. Wong, Cheng Zhang, Xinye Cao, Pedro Gimenes, George A. Constantinides, Wayne Luk, and Yiren Zhao. 2025. A3 : an Analytical Low-Rank Approximation Framework for Attention. arXiv:2505.12942 [cs.CL] https://arxiv.org/abs/2505.12942

[29] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. 2021. Nyströmformer: A Nyström-Based Algorithm for Approximating Self-Attention. arXiv:2102.03902 [cs.CL] https://arxiv.org/abs/2102.03902

[30] Yancheng Zhang, Mengxin Zheng, Xun Chen, Jingtong Hu, Weidong Shi, Lei Ju, Yan Solihin, and Qian Lou. 2025. zkVC: Fast Zero-Knowledge Proof for Private and Verifiable Computing. arXiv:2504.12217 [cs.CR] https://arxiv.org/abs/2504.12217

A MLP + LayerNorm Circuit Costs

A complete transformer layer consists of two sub-circuits: attention and the MLP + LayerNorm block. While the main text focuses on the attention sub-circuit—the target of our zkDietFormer optimizations—we additionally measured the cost of proving the MLP + LayerNorm sub-circuit, whose constraint count is the same for BERT and Nyströmformer because the MLP and LayerNorm dimensions depend only on n and d, not on the attention mechanism. Table 4 reports per-layer average MLP + LayerNorm costs for the Tiny configuration (L=2); all timing and memory figures are the mean of the BERT and Nyström runs. These measurements show that the MLP + LayerNorm sub-circuit and the attention sub-circuit contribute comparable costs at n=1024: per-layer MLP + LayerNorm proving takes 516.62 s versus 629.29 s for Nyström -Freivalds attention. The MLP + LayerNorm block consumes 112.48 GB peak RAM at n=1024, making it a co-bottleneck alongside attention for memory-constrained environments.

For reference, proving a complete Tiny transformer layer (attention + MLP + LayerNorm) at n=1024 requires approximately 2,289.03 s of per-layer setup and 1,145.91 s of per-layer proving (∼57 min total per layer). We report these measurements primarily as a baseline for future work on scalable ZK proving of the full transformer layer.

Table 4: MLP + LayerNorm costs for the Tiny configuration (L=2, d=128).

			Commit (ms)					
n	Constr. (M)	Setup (s)	G16	Lasso	Prove (s)	Verify (ms)	Proof Size (KB)	Peak RAM (GB)
128	0.98	94.23	119.3	147.8	28.38	285.0	755.8	26.66
256	1.96	189.21	217.8	170.3	58.79	311.3	849.9	52.30
512	3.92	385.63	441.3	306.0	137.28	405.5	1,026.2	101.93
1024	7.84	883.05	1,210.3	433.8	516.62	482.5	1,201.7	112.48